Reciproci Web Services

API Integration Documentation

Exchange Line & Commit Transaction APIs

Version 2.0 | March 2026

1. Overview

This document describes the integration rules and usage guidelines for two Reciproci loyalty web service APIs involved in product return and exchange sale transactions:

- **exchangeLine API — used to calculate the refund value for items being returned**
- **Commit Transaction API — used to commit the final return or exchange sale transaction**

Both APIs work together in a two-step flow. The exchangeLine API must be called first to retrieve tender and points data, and then the Commit API is called to finalize the transaction.

2. Exchange Line API

Endpoint (UAT): `https://bfl-uat-pos.reciproci.com/rprest/api/transaction/v1/exchangeLine`

HTTP Method: POST

2.1 Purpose

The exchangeLine API is called when a customer wishes to return one or more items from a prior purchase. It returns the tender details (refund amount and mode) and any points-related adjustments that must be factored into the final refund calculation at the POS or eCommerce system.

This API is applicable for both transaction types:

- Usual Return — customer returns item(s) with no new purchase
- Exchange Sale — customer returns item(s) and simultaneously purchases new item(s)

> **Important:** At this stage, Reciproci does not need to know whether the transaction will ultimately be a Return or an Exchange Sale. That distinction is resolved only at the Commit step.

2.2 Business Rules — Request

- Send only the items that are being returned. Do not include items that are NOT being returned, and do not include any newly purchased items.
- All item quantities must be sent as positive numbers (e.g. 1, 2, 3). Negative quantities are not accepted.
- `previousLineNo` must be the line number of the item in the original purchase transaction, used together with `previousReceiptNo` to uniquely identify the purchase transaction from which item being returned.
- `isReturn` must be set to **`"Yes"`** for every item sent in this request.

2.3 Business Rules — Response

- The tenderDetails node returns the amount to be refunded to the customer, per tender mode, based on the original purchase payment method.
- `totalRefundValue` — if the customer previously redeemed loyalty points and those points cannot be reversed (because they have since been used), this node contains the monetary value of those points. This amount must be deducted from the refund total at the POS/eCommerce system before returning money to the customer.

Calculation Example: If tenderDetails shows T1 = 200 AED, and totalRefundValue = 3 AED, then the net refund to the customer = 200 - 3 = 197 AED.

2.4 Worked Example

The following scenario illustrates end-to-end points handling across three transactions:

1. **Purchase (Transaction ID: ABC123)**

Customer buys 4 products for a total of 1,000 AED via cash (Tender Code: T1):

- Product-A: 100 AED
- Product-B: 200 AED
- Product-C: 300 AED
- Product-D: 400 AED

Customer accrues 1,500 points worth 15 AED.

2. **Subsequent Purchase (Transaction ID: XYZ234)**

Customer redeems all 1,500 points (worth 15 AED) on a new transaction. The points balance is now zero.

3. **Exchange Sale — Returning Product-B (200 AED)**

Customer returns Product-B. Points accrued on this item were 300 points worth 3 AED. Since these points have already been redeemed and cannot be reversed:

- tenderDetails: T1 = 200 AED
- totalRefundValue = 3 AED

Net refund to customer: 200 AED − 3 AED = 197 AED

2.5 Request Payload

Sample Request:

```
{
   "reqId": "BFLUAEREQ20260324_06",
   "storeId": "BFL-DXB",
   "terminalId": "1",
   "receiptNo": "BFLUAEREQ20260324_06",
   "reqTimeStamp": "2026-03-23 12:23:23",
   "cashierId": "EMP001",
   "channel": "POS",
   "memberId": 985,
   "commitRequestType": "Complete",
   "txnDate": "2026-03-24 12:23:23",
   "itemDetails": [
       {
           "itemType": "Product",
           "quantity": 1,
           "previousLineNo": 1,
           "isReturn": "Yes"
       }
   ],
   "previousReceiptNo": "BFLUAEREQ20260323_01"
}
```

2.6 Request Field Reference

Field	Type / Value	Description
reqId	String	Unique identifier for the request
storeId	String	Store identifier code
terminalId	String	POS terminal identifier
receiptNo	String	Receipt number for this transaction
reqTimeStamp	DateTime	Request timestamp (YYYY-MM-DD HH:mm:ss)
cashierId	String	Cashier employee identifier
channel	String	Transaction channel (e.g. POS)
memberId	Integer	Customer loyalty member ID
commitRequestType	String	Commit request type (e.g. Complete)
txnDate	DateTime	Transaction date and time
itemDetails	Array	List of items being returned (see below)
itemDetails[].itemType	String	Product type (e.g. Product)
itemDetails[].quantity	Integer	Quantity — must be a positive number (e.g. 1, 2, 3)
itemDetails[].previousLineNo	Integer	Line number of item in the original purchase transaction
itemDetails[].isReturn	String (Yes)	Must be set to 'Yes' for all items in this request
previousReceiptNo	String	Receipt number of the original purchase transaction being referenced

2.7 Response Payload

Sample Response (key fields shown):

```
{
    "reqId": "BFLUAEREQ20260324_06",
    "storeId": "BFL-DXB",
    "receiptNo": "BFLUAEREQ20260324_06",
    "memberId": 985,
    "itemDetails": [
        {
            "lineNo": 1,
            "itemType": "PRODUCT",
            "sku": "153831",
            "quantity": 1,
            "grossPrice": 200.0,
            "netPrice": 190.0,
            "vatAmount": 10.0,
            "previousLineNo": 1,
            "isReturn": "No",
            "basePointsAccrued": 300,
            "basePointsAccruedValue": 3.00
        }
    ],
    "tenderDetails": [
        { "code": "T1", "amount": 200.0 }
    ],
    "totalRefundValue": 3.0,
    "status": "Success"
}
```

2.8 Response Field Reference

Field	Type / Value	Description
`itemDetails[].basePointsAccrued`	Integer	Base loyalty points accrued on the returned item
`itemDetails[].basePointsAccruedValue`	Decimal	Monetary value of base points accrued (e.g. 3.00 AED)
`itemDetails[].brandPointsAccrued`	Integer	Brand loyalty points accrued (0 if none)
`itemDetails[].isReturn`	String	Reflects the isReturn flag sent in the request
`tenderDetails[].code`	String	Tender mode code from the original purchase (e.g. T1 = Cash)
`tenderDetails[].amount`	Decimal	Amount to be returned to customer via that tender mode
`totalRefundValue`	Decimal	Points accrued value to be deducted from refund amount. Applies when customer has already redeemed earned points and reversal is not possible.
`billAmountData.subTotal`	Decimal	Gross value of the returned item

Field	Type / Value	Description
`billAmountData.netPayableAmount`	Decimal	Net payable amount after discount
`pointsSummary`	Array	Updated member points balance after processing return
`status`	String	Response status: 'Success' or error

3. Commit Transaction API

Endpoint (UAT): `https://bfl-uat-pos.reciproci.com/rprest/api/transaction/v1/commitTransaction`
HTTP Method: POST

3.1 Purpose

The Commit Transaction API finalizes the return or exchange sale. It accepts the full item list (returned and/or newly purchased items), tender details, and the previous receipt reference, then records the transaction in the Reciproci system and updates the member's loyalty balance accordingly.

3.2 How Reciproci Classifies the Transaction

Field	Type / Value	Description
`Return`	previousReceiptNo is present AND all items have isReturn: Yes	
`Exchange Sale`	previousReceiptNo is present AND at least one item has isReturn: Yes AND at least one item has isReturn: No	

3.3 Business Rules — Request

3.3.1 Item Details

- All items — both returned and newly purchased — must be included in the single itemDetails array.
- `isReturn: "Yes"` marks an item as returned from the original purchase.
- `isReturn: "No"` marks an item as a new purchase in this transaction.
- Quantities must always be positive. The isReturn flag (not a negative quantity) signals a return.
- `previousLineNo` must be sent for items marked isReturn: "Yes", and it must exactly match the line number from the original purchase transaction. A mismatch will result in an API error.
- `previousReceiptNo` must match a valid purchase transaction receipt number present in the Reciproci system.

3.3.2 Tender Details

Tender details are not validated by Reciproci for financial accuracy — only the tender code (must exist in master data) and the format of the amount are validated. It is the responsibility of the POS/eCommerce system to ensure correct values.

For Exchange Sale transactions, tenders must be structured as follows:

- The net refund amount (after deducting totalRefundValue from exchangeLine response) must be sent as a Credit Note tender (e.g. Paper Credit Note, Tender Code: T32).
- Any remaining amount due for newly purchased items must be sent via the appropriate tender mode (e.g. Cash, Card).

Example: Returned item: 200 AED (minus 3 AED totalRefundValue = 197 AED net refund). New purchase: 500 AED. Tender: Credit Note (T32) = 197 AED + Cash (T1) = 303 AED. Total = 500 AED.

3.3.3 Points Accrual

For Exchange Sale transactions, points are accrued on the total value of newly purchased items. Example: if 500 AED of new items are purchased and the accrual rate is 1%, then 500 points are accrued (worth 5 AED at a burn rate of 1 point = 0.01 AED).

3.4 Request Payload

Sample Request — Exchange Sale:

```
{
    "reqId": "BFLUAEREQ20260324_06",
    "storeId": "BFL-DXB",
    "receiptNo": "BFLUAEREQ20260324_06",
    "memberId": 985,
    "language": "EN",
    "couponCodes": [],
    "itemDetails": [
        {
            "lineNo": 3,
            "sku": "153831",
            "quantity": 1,
            "grossPrice": 200.0,
            "netPrice": 190.0,
            "vatAmount": 10.0,
            "previousLineNo": 1,
            "isReturn": "Yes"       // Returned item
        },
        {
            "lineNo": 4,
            "sku": "153832",
            "quantity": 1,
            "grossPrice": 500.0,
            "netPrice": 475.0,
            "vatAmount": 25.0,
            "previousLineNo": null,
            "isReturn": "No"        // New purchase
        }
    ],
    "previousReceiptNo": "BFLUAEREQ20260323_01",
    "tenderDetails": [
        { "code": "T32", "amount": 197.0 },   // Paper Credit Note
        { "code": "T1",  "amount": 303.0 }    // Cash
```

```
    ]
}
```

3.5 Request Field Reference

Field	Type / Value	Description
reqId	String	Unique request identifier (should match exchangeLine reqId for exchange flow)
storeId	String	Store identifier code
receiptNo	String	Receipt number for this commit transaction
reqTimeStamp	DateTime	Request timestamp
terminalId	String	POS terminal identifier
channel	String	Transaction channel
language	String	Language code: EN or AR
memberId	Integer	Customer loyalty member ID
commitRequestType	String	Commit type (e.g. Complete)
txnDate	DateTime	Transaction date and time
couponCodes	Array	List of coupon codes (empty array if none)
itemDetails	Array	Full list of both returned and newly purchased items
itemDetails[].lineNo	Integer	Line number for this item in the current transaction
itemDetails[].itemType	String	Product type
itemDetails[].sku	String	Product SKU code
itemDetails[].quantity	Integer	Must be positive; use isReturn flag to indicate return
itemDetails[].grossPrice	Decimal	Item gross price
itemDetails[].discountAmount	Decimal	Discount applied to the item
itemDetails[].netPrice	Decimal	Net item price after discount
itemDetails[].vatAmount	Decimal	VAT amount on the item
itemDetails[].previousLineNo	Integer	For returned items: line number from original purchase. Leave blank for new purchases.
itemDetails[].isReturn	String (Yes/No)	'Yes' for returned items; 'No' for newly purchased items
previousReceiptNo	String	Receipt number of the original purchase transaction. Must match a valid transaction in the Reciproci system.
tenderDetails	Array	List of tender entries covering the net amount due

Field	Type / Value	Description
tenderDetails[].code	String	Tender mode code (e.g. T1 = Cash, T32 = Credit Note / Paper Credit Note)
tenderDetails[].amount	Decimal	Amount for this tender mode

3.6 Response Payload

Sample Response:

```
{
    "reqId": "BFLUAEREQ20260324_06",
    "status": "Success",
    "statusDetails": [
        { "code": 100, "message": "Transaction committed successfully" }
    ],
    "itemDetails": [
        { "lineNo": 3, "isReturn": "Yes", "basePointsAccrued": 300, "basePointsAccruedValue":
3.00 },
        { "lineNo": 4, "isReturn": "No",  "basePointsAccrued": 500, "basePointsAccruedValue":
5.00 }
    ],
    "pointsSummary": [
        { "pointsValue": 571.65, "points": 57165, "pointsType": "BASE" }
    ],
    "totalRefundValue": 0.0,
    "ttReferrenceNumber": "66809651"
}
```

3.7 Response Field Reference

Field	Type / Value	Description
status	String	Response status: 'Success' or error
statusDetails[].code	Integer	Status code (100 = Transaction committed successfully)
statusDetails[].message	String	Human-readable status message
itemDetails[].isReturn	String	Confirms return flag per line item
itemDetails[].basePointsAccrued	Integer	Points accrued on each purchased item
itemDetails[].basePointsAccruedValue	Decimal	Monetary value of points accrued per item
pointsSummary	Array	Updated member points balance post-transaction
totalRefundValue	Decimal	0.0 for fully committed exchange transactions
ttReferrenceNumber	String	Reciproci internal transaction reference number

Field	Type / Value	Description
`previousReceiptNo`	String	Echoes the previousReceiptNo from the request

4. End-to-End Exchange Sale Flow

Step	Action	Key Data Sent	Key Data Received
1	Call exchangeLine API with items being returned	itemDetails (isReturn: Yes), previousReceiptNo	tenderDetails, totalRefundValue, pointsSummary
2	Calculate net refund at POS	tenderDetails.amount − totalRefundValue	Net refund amount (e.g. 197 AED)
3	Present exchange items and tender to customer	New items, tender split	—
4	Call Commit API with full transaction	All items (returned + new), previousReceiptNo, tenderDetails (Credit Note + other tender)	status: Success, updated pointsSummary

5. Key Rules Summary

Rule	Detail
Positive quantities only	Quantities must always be positive integers. Use isReturn flag to indicate a return.
exchangeLine — returned items only	Only send items being returned. Do not include items not being returned or new purchase items.
isReturn in exchangeLine	Must always be 'Yes' for all items in the exchangeLine request.
previousLineNo matching	Must match the exact line number from the original purchase transaction. Mismatch causes a Commit API error.
previousReceiptNo validation	Must match a valid purchase receipt in the Reciproci system.
totalRefundValue deduction	Must be deducted from the refund amount at POS/eCommerce before returning cash to the customer.
Credit Note tender for refund	The net refund amount (after totalRefundValue deduction) must be sent as a Credit Note tender in the Commit request.

Rule	Detail
Tender not validated for amounts	Reciproci validates only tender code and format — financial accuracy is the responsibility of the originating system.
Points on new purchases	For Exchange Sale, points are accrued on the total value of newly purchased items only.